Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине

Цифровая аналитика

Курсовая работа

«Разработка программы с помощью нейронных сетей»

Выполнил:
студент группы 5130201/40003
Гумницкий К. С.
Принял:
В. А. Мулюха,
«____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1 Введение 3

2 Постановка задачи 4

2.1 Требования к программе 4

2.2 Требования к интерфейсу 5

3 Реализация 6

3.1 Взаимодействие с нейросетью 6

3.2 Работа с программой 8

4 Заключение 11

5 Список литературы 11

1 Введение

В настоящее время нейронные сети постоянно и быстро развиваются. Большие языковые модели, такие как DeepSeek или ChatGPT способны решать множество задач, начиная с поиска и анализа информации, советов по решению проблем пользователя и заканчивая генерацией программного кода по описанию задачи.

Цель данной работы — научиться пользоваться нейронными сетями для разработки программного обеспечения.

2 Постановка задачи

2.1 Требования к программе

Необходимо разработать программу для любительской астрономии, используя для программирования исключительно нейронные сети. Программа должна по введённым дате, времени и геолокации выдавать информацию о небесных объектах, которые можно будет увидеть в данный момент.

Пользователь вводит дату и время, программа считывает геолокацию. По этим данным она вычисляет и составляет карту звёздного неба. Программа использует географические координаты (градусы широты и долготы). Параметр высоты не важен, так как он влияет лишь на угол обзора и величину засветки от сторонних источников света. Геолокация определяется автоматически при запуске программы по IP-адресу. Программа также предусматривает ручной ввод координат.

Каталог небесных тел хранится в JSON-файле. Каждый объект имеет следующие характеристики:

- Название или код объекта (хранится в файле);
- тип объекта, его подтип (хранится в файле);
- склонение (хранится в файле);
- видимые размеры (хранится в файле);
- видимая звёздная величина (хранится в файле);
- высота (вычисляется программой);
- азимут (вычисляется программой).

Программа реализована на C++ с использованием различных библиотек для астрономических исследований. База данных берётся из открытых источников. Графика была сделана в QT.

2.2 Требования к интерфейсу

Программа отображает геопозицию и время, позволяет с помощью отдельных виджетов. Ниже отображается описание выделенного объекта, ещё ниже - список объектов, которые видны на данный момент на текущей геопозиции. Справа, на всё оставшееся пространство, отображается карта неба. Это круг, центр которого находится прямо "над головой"пользователя. На карту наносятся объекты. Ниже приведён примерный макет интерфейса (рис. 1).

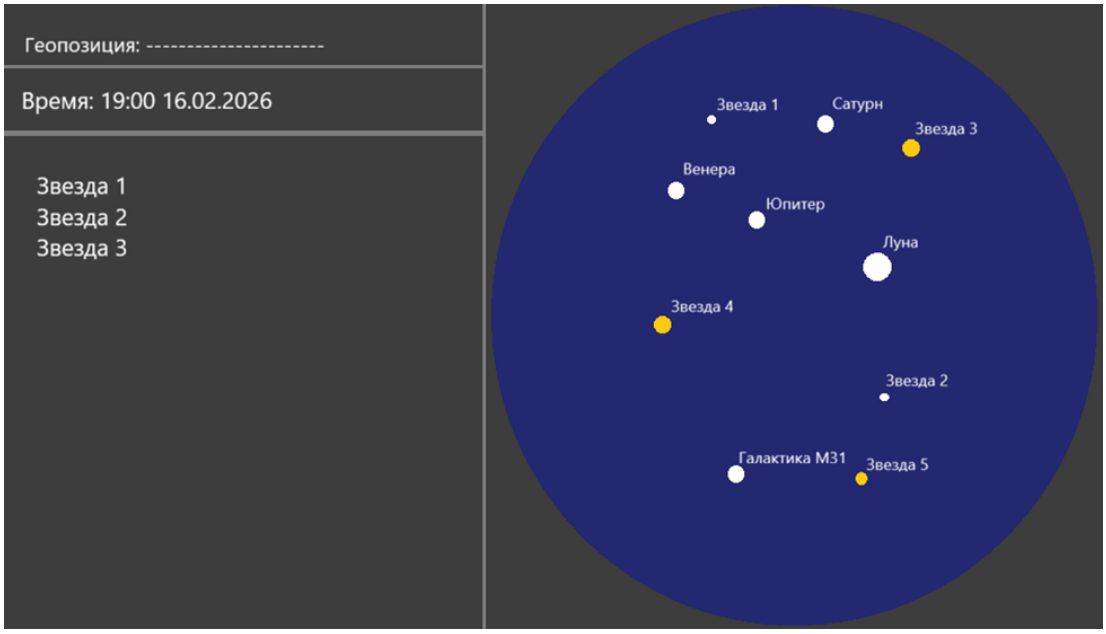

Рис. 1: Примерный макет интерфейса.

3 Реализация

3.1 Взаимодействие с нейросетью

Во время разработки программы использовалась нейросеть от компании DeepSeek версии DeepSeek-V3.2 . В самом начале модель получила техническое задание вместе с дополнительными пояснениями о системе сборки и требованиями к реализации (рис. 2).

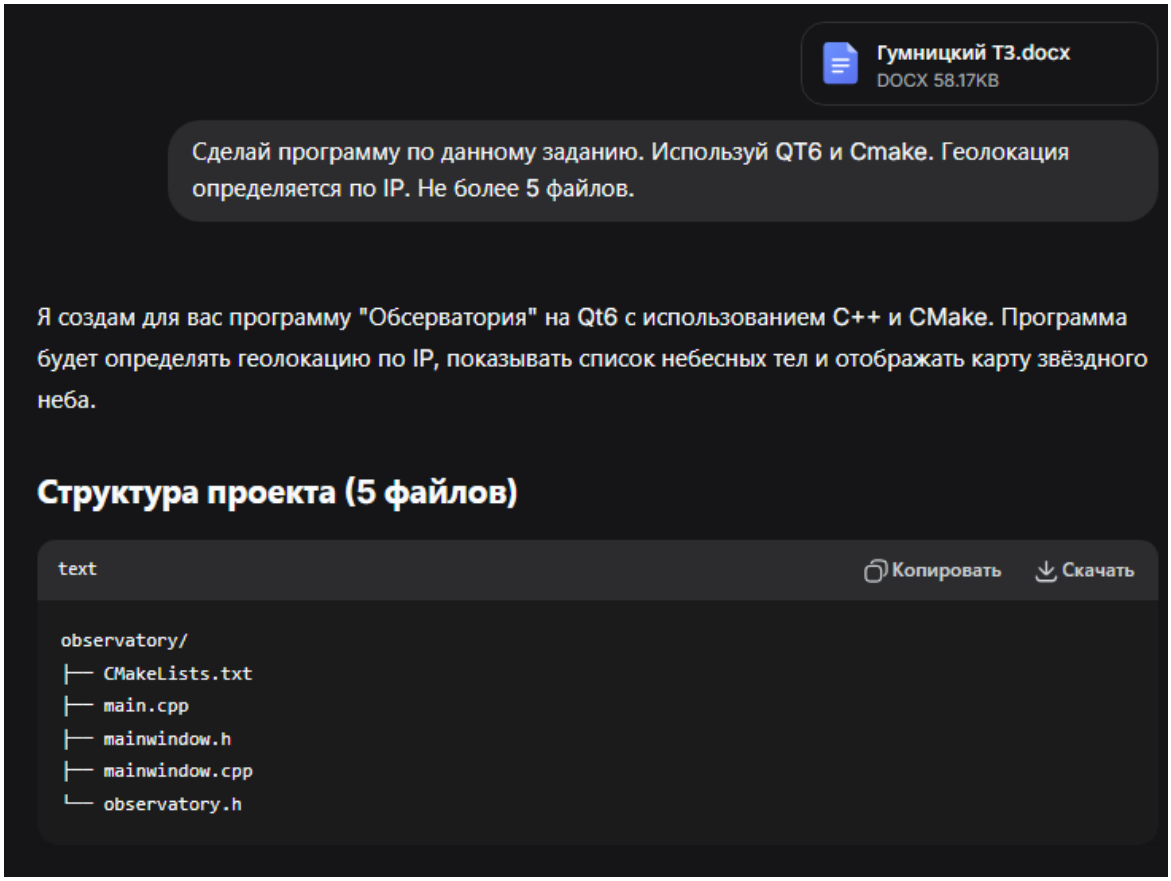

Рис. 2: Первый запрос.

Все дальнейшие запросы были нужны для исправления ошибок, редактирования пользовательского интерфейса и изменения некоторых ключевых алгоритмов программы. Информация об ошибках подавалась в виде текста об ошибке или снимка экрана из компилятора. Ниже приведён пример запроса

с исправлением интерфейса (рис. 3) и работы над ошибками (рис. 4).

Переработай карту. Размер объектов должен соответствовать их угловым размерам. Цвет от синего до белого - яркости. Выводи объекты, которые видны на небосводе. Пример - Stellarium

Вот полностью переработанный `mainwindow.cpp` с картой в стиле Stellarium:

cpp Копировать Скачать

```cpp
#include "mainwindow.h"
#include <QMessageBox>
#include <QHeaderView>
#include <QFont>
#include <QBrush>
#include <QColor>
#include <QDebug>
#include <QScrollArea>
#include <QtMath>
#include <QLinearGradient>

// ============= SkyMapWidget Implementation =============
SkyMapWidget::SkyMapWidget(QWidget *parent) : QWidget(parent) {
    setMinimumSize(700, 600);
    setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    setStyleSheet("background-color: black; border: 2px solid #444; border-radius: 8px;");
    m_hasSelection = false;
    m_showLabels = true;
}
```

Рис. 3: Редактирование интерфейса.

D:\QT\Observ\mainwindow.cpp:332: ошибка: 'class Observatory' has no member named 'getCatalogSize'
D:/QT/Observ/mainwindow.cpp: In member function 'void MainWindow::onCatalogLoaded()':
D:/QT/Observ/mainwindow.cpp:332:37: error: 'class Observatory' has no member named 'getCatalogSize'

```
 332 |    int catalogSize = m_observatory.getCatalogSize();
     |                                    ^~~~~~~~~~~~~~
```

Ошибка возникает потому что в классе `Observatory` нет метода `getCatalogSize()`. Нужно добавить этот метод. Вот исправления:

1. observatory.h (добавляем метод)

cpp Копировать Скачать

```cpp
// Добавьте эту строку в public раздел класса Observatory:
int getCatalogSize() const { return m_catalog.size(); }
```

Рис. 4: Исправление ошибки.

JSON-файл также был сгенерирован (рис. 5). В него были отобраны только те объекты, которые возможно увидеть в любительский телескоп.

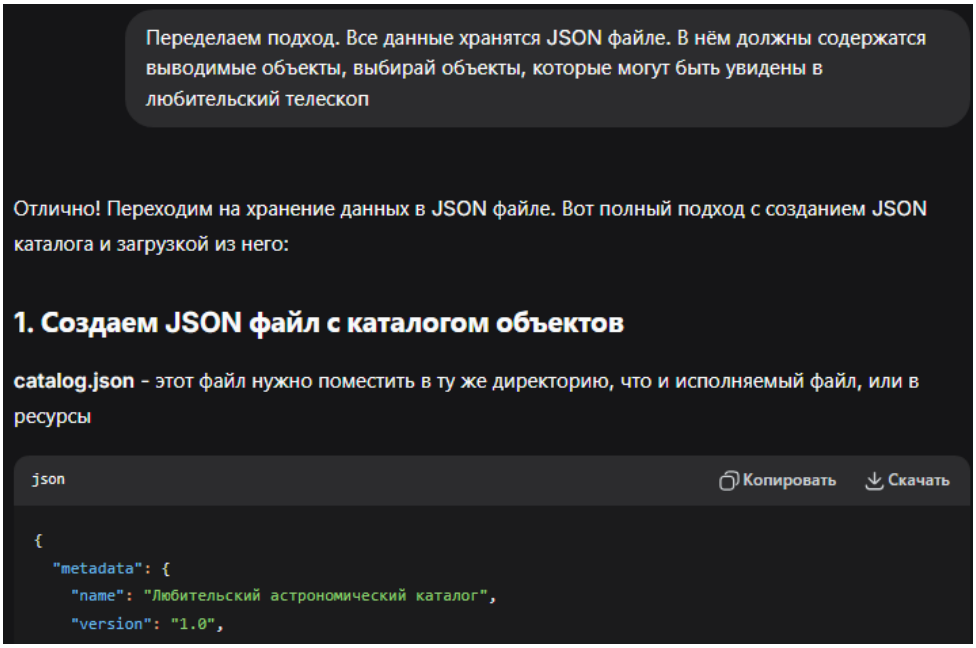

Рис. 5: Генерация JSON-файла.

3.2 Работа с программой

При запуске программа считывает системное время компьютера и геопозицию пользователя. Для работы приложения необходимо подключение к интернету. После завершения вычислений пользователь видит основной интерфейс программы 6)

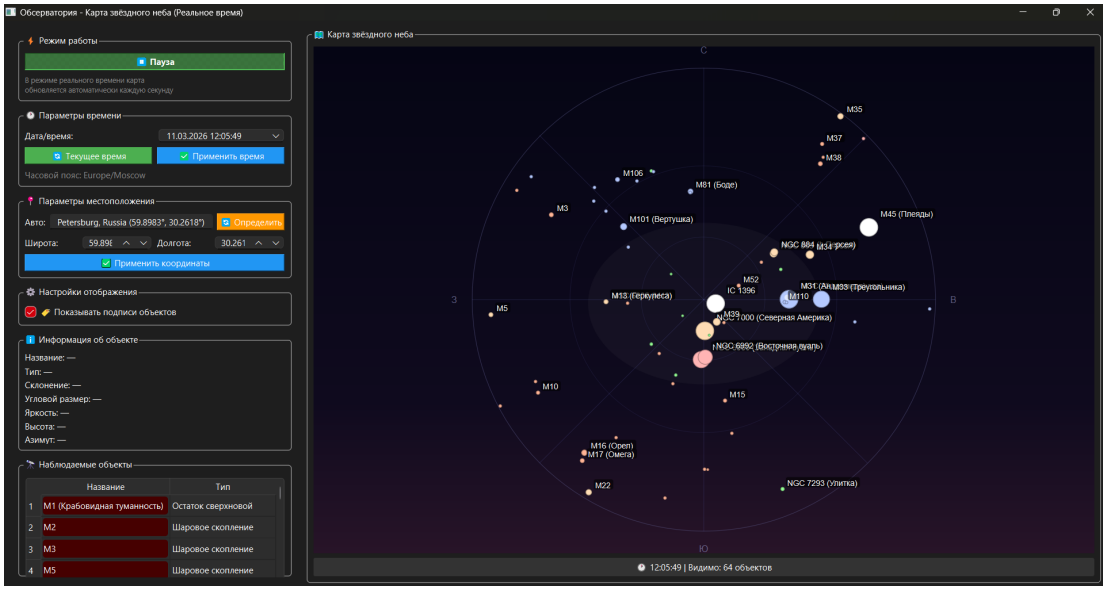

Рис. 6: Интерфейс программы.

На экране находятся следующие инструменты (сверху-вниз и вправо):

1. **Режим работы**. Эта область регулирует изменение положения объектов в настоящее время. Если оно включено, объекты будут менять свою высоту и азимут в реальном времени.

2. **Параметры времени**. В этой области можно ввести собственную дату и время. Для этого надо изменить дату или время и нажать на кнопку "Применить время". Для возврата к настоящему времени надо воспользоваться кнопкой "Текущее время".

3. **Параметры местоположения**. В этой области можно ввести другую геолокацию. Для этого нужно изменить изменить широту или долготу и нажать на кнопку "Применить координаты". Для возврата к пользовательским координатам на нажать на кнопку "Определить".

4. **Настройки отображения**. Можно включить или выключить подписи на карте.

5. **Информация об объекте**. характеристики выбранного в таблице объекта.

6. **Наблюдаемые объекты**. Таблица со всеми объектами, наблюдаемые на текущей позиции в это время.

7. **Карта звёздного неба**. Карта - это окружность с отмеченными сторонами света, внутри которого находятся видимые объекты. Их положение устанавливается параметрами высота и азимут, которые вычисляются в программе.

Ниже приведён скриншот программы после изменения всех параметров (рис. 7).

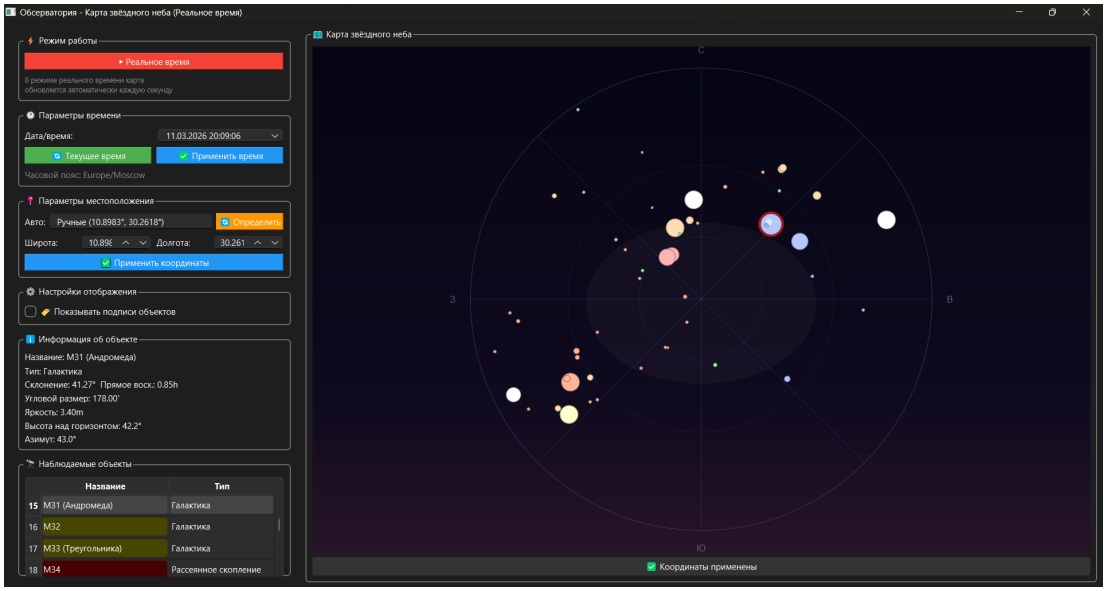

Рис. 7: Интерфейс программы после изменений.

4 Заключение

В результате работы была создана программа "Обсерватория"для любительской астрономии. Программа корректно вычисляет положение небесных тел и формирует на основе вычисленных данных карту неба. Программа корректно считывает время и позицию пользователя, позволяет их изменять. Весь процесс работы над приложением проходило в web-версии нейронной сети DeepSeek.

Исходный код приложения, техническое задание и диалог с языковой моделью можно посмотреть в публичном репозитории на GitHub, доступному по данной ссылке:

https://github.com/Nebulaner/Observatory.

Ссылка на чат с нейронной сетью:

https://chat.deepseek.com/share/lmw1xb1v828tmd3uf5

5 Список литературы

[1] GitHub: https://github.com.

[2] DeepSeek: https://deepseek.com.